Software Requirements Specification (SRS)

Wrong-Side Driving Detection System

---

1. Introduction

1.1 Purpose of the System

The purpose of this system is to detect vehicles driving in the wrong direction on highways using AI and Computer Vision techniques. The system aims to enhance road safety by automatically identifying violations and notifying authorities with visual evidence.

1.2 Scope of the System

This system processes CCTV footage (live or recorded), detects vehicles, tracks their movement, identifies wrong-way driving, and generates alerts with evidence. It includes an edge detection module, backend API, and web dashboard.

1.3 Definitions and Abbreviations

- AI: Artificial Intelligence
- CV: Computer Vision
- ROI: Region of Interest
- API: Application Programming Interface
- CCTV: Closed Circuit Television

1.4 Intended Users

- Traffic Police Authorities
- City Surveillance Teams
- System Administrators

---

2. Overall Description

2.1 System Overview

The system is a microservices-based architecture consisting of:

- Edge Node (AI processing)
- Backend API (data handling)
- Web Dashboard (user interface)

2.2 User Categories

- Admin: Full control over system and cameras
- Operator: Monitor violations and view evidence

2.3 Operating Environment

- OS: Linux/Windows
- Backend: Python (FastAPI)
- Frontend: React (Browser-based)
- Hardware: CCTV Cameras, CPU-based systems

2.4 Assumptions and Constraints

- Cameras are fixed/static
- Clear view of road lanes
- Stable network connection for API communication

---

3. Functional Requirements

3.1 User Registration and Login

FR1: The system shall allow admin login.
FR2: The system shall authenticate users securely.

3.2 Video Processing

FR3: The system shall process live CCTV feeds.
FR4: The system shall support recorded video input.

3.3 Vehicle Detection and Tracking

FR5: The system shall detect vehicles using AI models.
FR6: The system shall assign unique IDs to tracked vehicles.

3.4 Wrong-Way Detection

FR7: The system shall calculate motion vectors of vehicles.
FR8: The system shall identify direction based on displacement.
FR9: The system shall detect violations after N consecutive frames.

3.5 Evidence Capture

FR10: The system shall store a 10-second video clip.
FR11: The system shall capture image snapshots.

3.6 Backend Communication

FR12: The system shall send violation data to backend API.
FR13: The system shall store metadata.

3.7 Dashboard Display

FR14: The system shall display violations on dashboard.
FR15: The system shall show image, timestamp, and video.

---

4. Non-Functional Requirements

4.1 Reusability

- Modular microservices architecture
- Independent components (AI, backend, frontend)

4.2 Maintainability

- Clean code structure
- Easy model updates and API changes

4.3 Reliability

- Continuous monitoring
- Accurate detection with minimal false positives

4.4 Security

- Authentication for dashboard
- Secure API endpoints

---

5. System Constraints

5.1 Hardware Limitations

- Depends on CPU/GPU performance

5.2 Software Restrictions

- Requires Python environment and dependencies

5.3 Regulatory Policies

- Must comply with local surveillance and privacy laws

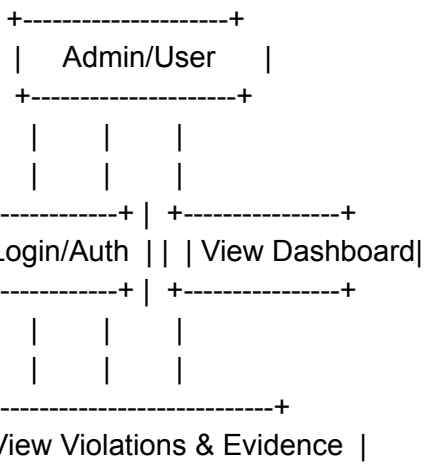
System Design Support for Quality Attributes

- Reusability: Components can be reused in other traffic systems
- Maintainability: Microservices allow independent updates
- Reliability: Motion-based detection works in poor conditions
- Security: Controlled access and secure data handling

---

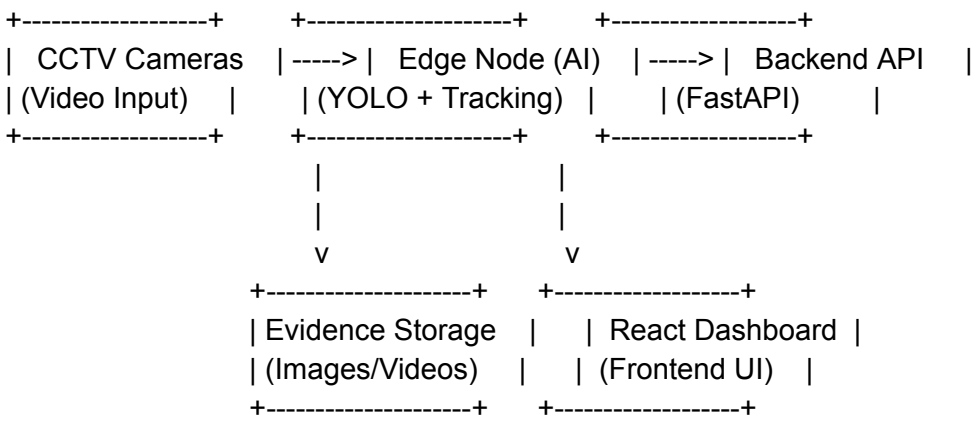
6. UML Diagrams

6.1 Use Case Diagram



+----------------------------+

6.2 System Architecture Diagram

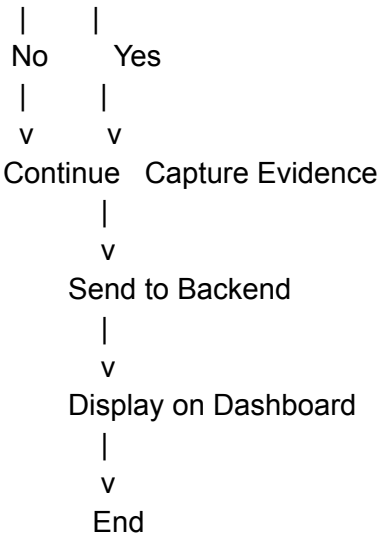


6.3 Sequence Diagram

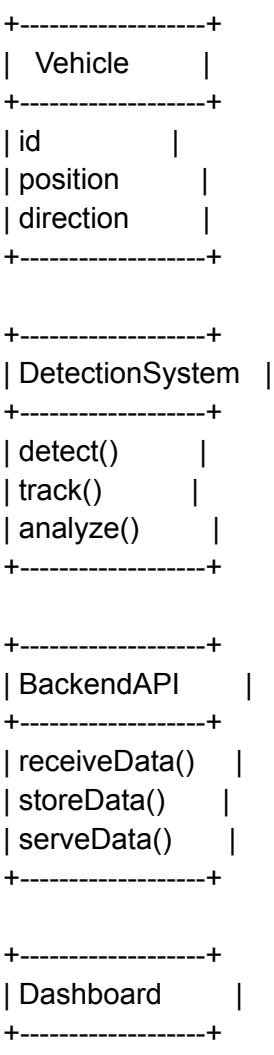
User -> Dashboard: Open Dashboard
Dashboard -> Backend: Request Violations
Backend -> Edge Node: Fetch Latest Data
Edge Node -> Backend: Send Violation JSON
Backend -> Dashboard: Return Data
Dashboard -> User: Display Violations

6.4 Activity Diagram

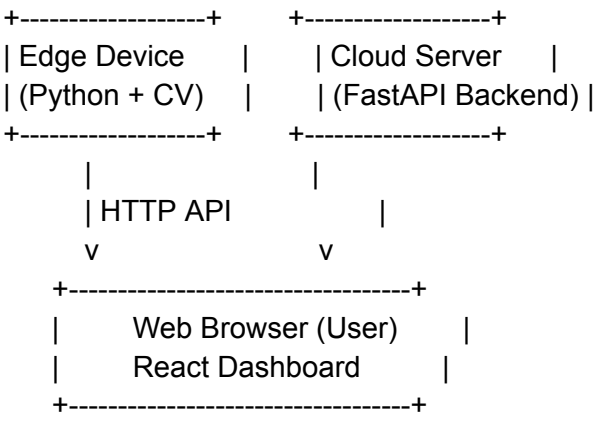
Start
|
v
Capture Video Feed
|
v
Detect Vehicles
|
v
Track Movement
|
v
Analyze Direction
|
v
Is Wrong Direction?



6.5 Class Diagram



6.6 Deployment Diagram



Conclusion

This system provides a scalable, efficient, and intelligent solution for detecting wrong-side driving using AI and computer vision, improving traffic monitoring and public safety.